

This session we're going to cover:

- For loop
- Range
- Comprehensions

حلقه for در پایتون

در پایتون از دستور `for`

برای پیمایش هر شیء قابل تکرار از جمله دنباله‌هایی مانند لیست‌ها، رشته‌ها و دیکشنری‌ها، استفاده می‌شود

For loop Syntax:

```
for item in sequence:  
    # Run this block of code
```

```
In [12]: languages = ['Swift', 'Python', 'Go']
```

```
# start of the loop  
for lang in languages:  
    print(lang)  
    print('-----')  
# end of the for loop  
  
print('Last statement')
```

```
Swift  
-----  
Python  
-----  
Go  
-----  
Last statement
```

```
In [13]: language = 'Python'  
# iterate over each character in language  
for x in language:  
    print(x)
```

```
P  
y  
t  
h  
o  
n
```

```
In [1]: my_list = [1, 2, 3, 4]  
for i in my_list:  
    print(f'{i} * 2 = {i*2}')
```

```
1 * 2 = 2
2 * 2 = 4
3 * 2 = 6
4 * 2 = 8
```

```
In [2]: people = (('tom', 23), ('peter', 32), ('stewie', 9))
        for person in people:
            print(person)
```

```
('tom', 23)
('peter', 32)
('stewie', 9)
```

```
In [3]: for name, age in people:
        print(f'{name} is {age} years old.')
```

```
tom is 23 years old.
peter is 32 years old.
stewie is 9 years old.
```

```
In [4]: people = {'peter':(34, 160), 'tom':(40, 170), 'brian':(10, 110)}
        for person in people:
            print(person, people[person])
```

```
peter (34, 160)
tom (40, 170)
brian (10, 110)
```

از enumerate()

for استفاده می‌شود تا در حین پیمایش، به اندیس و مقدار هر آیتم دسترسی پیدا کنیم همراه با حلقه

```
In [16]: b = ["eat", "sleep", "repeat"]
        for index, value in enumerate(b):
            print (index, value)
```

```
0 eat
1 sleep
2 repeat
```

حلقه‌های تو در تو (Nested for loop)

یک حلقه همچنین می‌تواند شامل یک حلقه دیگر در داخل خود باشد. به این حلقه‌ها، حلقه‌های تو در تو گفته می‌شود

در یک حلقه تو در تو، حلقه داخلی به ازای هر تکرار حلقه بیرونی، یک بار اجرا می‌شود

```
In [16]: # outer loop
        attributes = ['Electric', 'Fast']
        cars = ['Tesla', 'Porsche', 'Mercedes']

        for attribute in attributes:
            for car in cars:
```

```
print(attribute, car)

# this statement is outside the inner loop
print("-----")
```

```
Electric Tesla
Electric Porsche
Electric Mercedes
-----
Fast Tesla
Fast Porsche
Fast Mercedes
-----
```

تابع `range()` دنباله‌ای از اعداد صحیح با فواصل یکسان تولید می‌کند

```
In [2]: range(10)
```

```
Out[2]: range(0, 10)
```

```
In [3]: list(range(10))
```

```
Out[3]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
In [4]: # A start, end, and step (which may be negative) can be given
```

```
first_list = list(range(0, 20, 2))

second_list = list(range(5, 0, -1))

print(first_list)
print(second_list)
```

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
[5, 4, 3, 2, 1]
```

```
In [5]: for i in range(0, 10, 2):
        print(i)
```

```
0
2
4
6
8
```

Exercise:

Expected output:

range(1, 20) if the value is even, add to even_list, else in odd_list, count the total of each lists:

- even_list = [2, 4, 6, 8, 10, 12, 14, 16, 18], total = 9
- odd_list = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19], total = 10

```
In [6]: count_odd = 0
count_even = 0
even_list = []
odd_list = []
for i in range(1, 20):
    if i % 2 == 0:
        even_list.append(i)
        count_even += 1
    else:
        odd_list.append(i)
        count_odd += 1

print(f'even_list = {even_list}, total = {count_even}')
print(f'odd_list = {odd_list}, total = {count_odd}')
```

```
even_list = [2, 4, 6, 8, 10, 12, 14, 16, 18], total = 9
odd_list = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19], total = 10
```

```
In [5]: for i in range(4):
        for j in range(4):
            print((i, j))
```

```
(0, 0)
(0, 1)
(0, 2)
(0, 3)
(1, 0)
(1, 1)
(1, 2)
(1, 3)
(2, 0)
(2, 1)
(2, 2)
(2, 3)
(3, 0)
(3, 1)
(3, 2)
(3, 3)
```

Challenge:

ساخت جدول ضرب

```
In [23]: for i in range(10):
        for j in range(10):
            print(i, j, end=' ')
        print()
```

```

0 0 0 1 0 2 0 3 0 4 0 5 0 6 0 7 0 8 0 9
1 0 1 1 1 2 1 3 1 4 1 5 1 6 1 7 1 8 1 9
2 0 2 1 2 2 2 3 2 4 2 5 2 6 2 7 2 8 2 9
3 0 3 1 3 2 3 3 3 4 3 5 3 6 3 7 3 8 3 9
4 0 4 1 4 2 4 3 4 4 4 5 4 6 4 7 4 8 4 9
5 0 5 1 5 2 5 3 5 4 5 5 5 6 5 7 5 8 5 9
6 0 6 1 6 2 6 3 6 4 6 5 6 6 6 7 6 8 6 9
7 0 7 1 7 2 7 3 7 4 7 5 7 6 7 7 7 8 7 9
8 0 8 1 8 2 8 3 8 4 8 5 8 6 8 7 8 8 8 9
9 0 9 1 9 2 9 3 9 4 9 5 9 6 9 7 9 8 9 9

```

```

In [27]: rows = int(input("Enter rows:"))
        cols = int(input("Enter Cols:"))

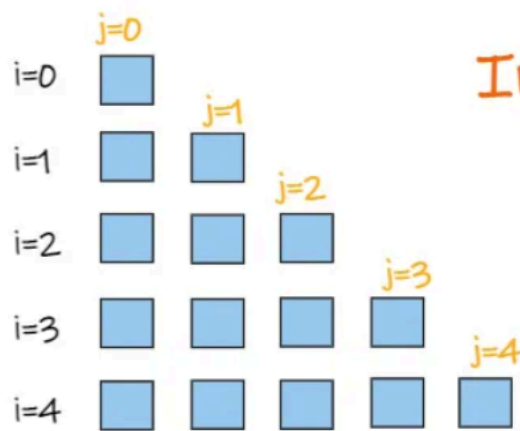
        for i in range(0, rows):
            for j in range(0, cols):
                print("*", end="")
            print()

```

```

*****
*****
*****
*****
*****

```



Increasing Triangle

we will print the character row by row.....

```

for i in range(n):
    for j in range(i+1):

```

print n rows, so i runs from 0 to n
and
print (i+1) number of columns.
so j runs from 0 to (i+1)

➡ for i in range(n)

➡ for j in range(i+1)

```
In [2]: n = 5
for i in range(n):
    for j in range(i + 1):
        print('*', end=' ')
    print()
```

```
*
* *
* * *
* * * *
* * * * *
```

```
In [3]: n = 5
for i in range(n):
    for j in range(i, n):
        print('*', end=' ')
    print()
```

```
* * * * *
* * * *
* * *
* *
*
```

```
In [28]: rows = 5
for i in range(1, rows + 1):
    print(' ' * (rows - i) + '*' * (2 * i - 1))
```

```

*
***
*****
*****
*****
```

```
In [4]: n = 5
p = 0

for i in range(n):
    p = 1
    for j in range(i + 1):
        print(p, end=' ')
        p += 1 # p = p + 1
    print()
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

سیستم حقوق و دستمزد شرکت

شما به عنوان توسعه‌دهنده تازه‌کار پایتون در یک شرکت کوچک استخدام شده‌اید

هر ماه، واحد منابع انسانی فهرستی از کارکنان و تعداد ساعات کاری آن‌ها را به شما می‌دهد

وظیفه شما محاسبه حقوق ماهانه هر کارمند و چاپ گزارش حقوق و دستمزد است

- `employees = ["Alice", "Bob", "Charlie", "David", "Emma"]`
- `hours_worked = [160, 145, 172, 150, 168]`
- `hourly_rate = 20 ==>>> salary = hours * hourly_rate`

Example output:

- Employee: Alice
- Hours Worked: 160
- Salary: \$3200

```
In [2]: employees = ["Alice", "Bob", "Charlie", "David", "Emma"]
hours_worked = [160, 145, 172, 150, 168]

hourly_rate = 20

for i in range(len(employees)):
    employee = employees[i]
    hours = hours_worked[i]
    salary = hours * hourly_rate

    print(f"Employee: {employee}")
    print(f"Hours Worked: {hours}")
    print(f"Salary: ${salary}")
    print("-" * 30)
```

```
Employee: Alice
Hours Worked: 160
Salary: $3200
-----
Employee: Bob
Hours Worked: 145
Salary: $2900
-----
Employee: Charlie
Hours Worked: 172
Salary: $3440
-----
Employee: David
Hours Worked: 150
Salary: $3000
-----
Employee: Emma
Hours Worked: 168
Salary: $3360
-----
```

List Comprehensions

Syntax: `[expr for value in collection if condition]`

```
In [12]: a = [2, 3, 4, 5]
res = [val ** 2 for val in a]
print(res)
```

[4, 9, 16, 25]

```
In [13]: a = [1, 2, 3, 4, 5]
res = [val for val in a if val % 2 == 0]
print(res)
```

[2, 4]

```
In [11]: strings = ["a", "as", "bat", "car", "dove", "python"]
x = []
for string in strings:
    if len(string) > 2:
        x.append(string.upper())

print(x)
```

['BAT', 'CAR', 'DOVE', 'PYTHON']

```
In [7]: # Now we use List comprehension
strings = ["a", "as", "bat", "car", "dove", "python"]
result = [x.upper() for x in strings if len(x) > 2]
print(result)
```

['BAT', 'CAR', 'DOVE', 'PYTHON']

Dictionary comprehension

Syntax: {key: value for (key, value) in iterable if condition}

```
In [15]: sq = {x: x**2 for x in range(1, 6)}
print(sq)
```

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}